INFORME TÉCNICO

Migración de Cosmos Check-In a Microsoft Azure

Inventario de todo lo creado, configuración, seguridad, operación y pendientes | Fecha: 04/11/2026 | Uso interno: contiene identificadores de la suscripción (no contiene contraseñas)

1. Resumen ejecutivo

Se ha creado en Azure toda la infraestructura que sustituirá al alojamiento actual de Cosmos Check-In (panel web en Netlify y API en un servidor de AWS administrado con Coolify). El instalador terminó sin errores el **30/10/2026**: las APIs de producción y de QA están en marcha y responden a su comprobación de salud (`/health/ready`) en sus direcciones de Azure.

Qué está listo
Dos ambientes completos (producción y QA): base de datos, API, panel web, registro de imágenes, bóvedas de secretos, monitoreo, alertas, respaldos semanales, identidades sin contraseñas y despliegue continuo desde GitHub con vuelta atrás automática.

Qué falta para que los usuarios usen Azure
(1) Cambios DNS, que administra otra área: solicitud enviada aparte. (2) Primer despliegue del panel por GitHub. (3) Carga y validación de datos en QA. (4) Ventana de corte con migración de los datos de producción.
Hoy, **ningún usuario usa todavía Azure**: `checkin` y `apicheckin` siguen apuntando a Netlify y AWS.

2. Datos de la cuenta

Dato	Valor
Tenant (directorio)	COSMOS AGENCIA MARITIMA SAC (`8b333033-5e56-4a8b-af69-58ea178eb35b`)
Suscripción	Azure Subscription - Check-In (`c02ac35c-2fd4-4903-ad88-27a313620306`)
Tipo de suscripción	CSP (`CSP_2015-05-01`), facturada por el partner. Sin tope de gasto.
Cuenta de administración	`admin.azure@cosmos.com.pe` (objectId `a4844e87-b8da-435c-a3ca-5f6c859a77aa`), rol Owner en la suscripción desde el 28/10/2026 16:44 UTC
Región	East US 2 (`eastus2`)
Prefijo de nombres	`cosmosck` (se descartó `cosmosci` porque `acrcosmosciprod` estaba ocupado en Azure)
Etiquetas de los grupos	`proyecto=cosmos-checkin`, `ambiente=prod|qa`
Repositorio que despliega	GitHub `Cosmos-Marcacion/cosmos` (privado, monorepo)
Correo de alertas y soporte	`marcostorresalarcon@gmail.com`

3. Arquitectura

Pieza	Antes	En Azure
Panel web	Netlify (`checkincosmos.netlify.app`)	Azure Static Web Apps (prod Standard, QA Free)

API (NestJS)	Coolify en AWS (`98.87.37.45`)	Azure Container Apps, imagen Debian con LibreOffice y TensorFlow
Base de datos	MongoDB Atlas 8.0 (cluster compartido `focodigitalmongodb`, base `cosmos`, ~12 MB)	Azure DocumentDB (compatible con MongoDB): prod M10, QA Free
Secretos	Variables de Coolify	Azure Key Vault, uno por ambiente, leído por identidad administrada
Imágenes	Construcción en el servidor	Azure Container Registry (Basic)
Registros	Docker local	Log Analytics
Respaldos	Sin respaldo programado	Job semanal a Blob Storage, retención 90 días
Despliegue	Manual / Coolify	GitHub Actions con OIDC, gates de calidad y revisiones con vuelta atrás
Archivos subidos	AWS S3	**Siguen en AWS S3** (fuera del alcance). QA escribe bajo el prefijo `qa/`

Flujo: el navegador carga el panel desde Static Web Apps y este llama a la API por su dominio (`apicheckin.cosmos.com.pe`); la app móvil llama al mismo dominio. La API lee sus secretos de Key Vault y conecta con DocumentDB por TLS. El panel no usa el proxy `/api` de Static Web Apps porque corta a 45 s y la conversión de formatos a PDF admite hasta 120 s.

4. Inventario de recursos creados

4.1 Grupo `rg-cosmosck-prod` (producción y compartidos)

Nombre	Tipo	Configuración
`law-cosmosck`	Log Analytics workspace	Compartido por los dos entornos de Container Apps
`acrcosmosckprod`	Container Registry, Basic	Usuario admin desactivado; acceso solo por identidades (AcrPull/AcrPush). Repositorios: `cosmos-api` (etiqueta `inicial` y una por commit), `mongo-backup:1`. Lo usan ambos ambientes.
`kv-cosmosck-prod`	Key Vault	Autorización RBAC. Secretos en la sección 6.
`docdb-cosmosck-prod`	DocumentDB (mongoClusters)	Tier M10, 32 GB, MongoDB 7.0, 1 shard, sin alta disponibilidad, usuario `cosmosadmin`. Firewall: solo servicios de Azure.
`id-cosmosck-api-prod`	Identidad administrada	La usa la API de producción
`id-cosmosck-respaldo`	Identidad administrada	La usa el job de respaldo
`id-github-deploy`	Identidad administrada	La usa GitHub Actions por OIDC (clientId `11a7b490-1210-41a8-90b7-7bc882f63dd2`)
`swa-cosmosck-prod`	Static Web App, Standard	`polite-hill-09221710f.2.azurestaticapps.net`
`cae-cosmosck-prod`	Entorno de Container Apps	Dominio por defecto `grayforest-d8107249.eastus2.azurecontainerapps.io`
`ca-cosmosck-api-prod`	Container App	1 CPU / 2 GiB, 1 a 3 réplicas. Detalle en sección 5. Revisión inicial `ca-cosmosck-api-prod--kesv8bg`.
`stcosmosckrespaldos`	Storage Account, Standard_LRS	TLS 1.2 mínimo, sin acceso público. Contenedor `respaldos`; regla `borrar-90-dias`.

`job-respaldo-semanal`	Container Apps Job	Domingo 07:00 UTC (02:00 Lima), 0,5 CPU / 1 GiB, máx. 1 h, 1 reintento
`ag-cosmos`	Action Group	Envía correo al soporte
`api-errores-5xx`	Alerta de métrica	Más de 20 respuestas 5xx en 5 min (evaluación cada minuto), severidad 2
`api-reinicios`	Alerta de métrica	Más de 3 reinicios del contenedor en 15 min, severidad 2

4.2 Grupo `rg-cosmosck-qa` (pruebas)

Nombre	Tipo	Configuración
`kv-cosmosck-qa`	Key Vault	RBAC; la identidad de QA no puede leer secretos de producción
`docdb-cosmosck-qa`	DocumentDB	Tier Free, 32 GB, MongoDB 7.0. Se pausa tras 60 días sin conexiones (lo evita el job de ping).
`id-cosmosck-api-qa`	Identidad administrada	API de QA y job de ping
`swa-cosmosck-qa`	Static Web App, Free	`gray-tree-0eb73fe0f.1.azurestaticapps.net`
`cae-cosmosck-qa`	Entorno de Container Apps	Dominio por defecto `salmonmeadow-21349ce3.eastus2.azurecontainerapps.io`
`ca-cosmosck-api-qa`	Container App	1 CPU / 2 GiB, **0 a 3 réplicas** (se apaga sin tráfico). Revisión inicial `ca-cosmosck-api-qa--1g7bzac`.
`job-ping-qa`	Container Apps Job	Lunes 08:00 UTC, 0,25 CPU / 0,5 GiB, abre una conexión a la base de QA

4.3 Proveedores registrados en la suscripción

`Microsoft.App`, `Microsoft.OperationalInsights`, `Microsoft.ContainerRegistry`, `Microsoft.KeyVault`, `Microsoft.DocumentDB`, `Microsoft.Web`, `Microsoft.Storage`, `Microsoft.Insights`, `Microsoft.ManagedIdentity`.

4.4 Direcciones

Ambiente	Servicio	Dominio final	Dirección de Azure (activa hoy)
Producción	Panel	`checkin.cosmos.com.pe`	`polite-hill-09221710f.2.azurestaticapps.net`
Producción	API	`apicheckin.cosmos.com.pe`	`ca-cosmosck-api-prod.grayforest-d8107249.eastus2.azurecontainerapps.io`
QA	Panel	`qa.cosmos.com.pe`	`gray-tree-0eb73fe0f.1.azurestaticapps.net`
QA	API	`api-qa.cosmos.com.pe`	`ca-cosmosck-api-qa.salmonmeadow-21349ce3.eastus2.azurecontainerapps.io`

Los dominios de producción se conservan a propósito: los APK instalados llaman a `apicheckin` y las llaves de acceso WebAuthn están ligadas al dominio del panel. Cambiarlos obligaría a reinstalar la app y volver a registrar llaves.

5. Configuración de la API (Container Apps)

Parámetro	Valor

Recursos por réplica	1 CPU / 2 GiB (la guía proponía 0,5 / 1 GiB; con 1 GiB el contenedor muere por memoria al cargar TensorFlow y lanzar LibreOffice)
Escala	Prod: mínimo 1, máximo 3. QA: mínimo 0, máximo 3.
Ingress	Externo, HTTPS obligatorio (`allowInsecure=false`), puerto 3000
Modo de revisiones	**Múltiple**: cada cambio crea una revisión sin tráfico; solo recibe tráfico tras pasar `/healt h/ready`
Sonda de arranque	`/health`, cada 10 s, 10 fallos, espera inicial 10 s
Sonda de vida	`/health`, cada 30 s, 3 fallos
Sonda de disponibilidad	`/health/ready` (comprueba la base), cada 10 s, 3 fallos
Imagen inicial	`acrcosmosckprod.azurecr.io/cosmos-api:inicial` (después, una etiqueta por commit)
Acceso al registro	Por identidad administrada, sin usuario ni contraseña
Variables fijadas por el instalador	`NODE_ENV=production`, `PORT=3000`, `TZ=America/Lima`, `WEBAUTHN_RP_ID` y `WEBAUTHN_ORI GIN` (dominio del panel del ambiente), `CORS_EXTRA_ORIGINS` (dirección de Static Web Apps)
Resto de variables	Copiadas del `.env` exportado de Coolify. Las sensibles van a Key Vault (sección 6).
Diferencias de QA	Archivos en S3 bajo `qa/` (`AWS_S3_KEY_PREFIX`); todo correo desviado al soporte con el destinatario original en el asunto (`MAIL_REDIRECT_TO`); Visma compartido (solo lectura)

6. Seguridad

6.1 Secretos en Key Vault (solo nombres)

Secreto	Contenido	Bóvedas
`mongo-uri`	Cadena de conexión a DocumentDB (TLS, SCRAM-SHA-256)	prod y qa
`docdb-admin-password`	Contraseña de `cosmosadmin`, generada al crear el cluster	prod y qa
`jwt-secret`	Secreto JWT nuevo (48 bytes aleatorios). El anterior estaba en el repositorio y se retiró.	prod y qa
`password-cosmos`, `password-email`	Credenciales de la aplicación y del correo	prod y qa
`aws-access-key-id`, `aws-secret-a ccess-key`	Acceso a S3	prod y qa
`visma-password`	Integración con Visma	prod y qa
`acceso-instalador`	Marca de prueba de acceso del instalador (sin valor sensible)	prod y qa

Consecuencia del JWT nuevo: tras el corte, todos los usuarios deberán iniciar sesión de nuevo una vez.

6.2 Identidades y permisos

Identidad	Rol	Ámbito
`id-cosmosck-api-prod`	AcrPull	`acrcosmosckprod`
	Key Vault Secrets User	`kv-cosmosck-prod`

`id-cosmosck-api-qa`	AcrPull	`acrcosmosckprod`
	Key Vault Secrets User	`kv-cosmosck-qa`
`id-cosmosck-respaldo`	AcrPull	`acrcosmosckprod`
	Key Vault Secrets User	`kv-cosmosck-prod`
	Storage Blob Data Contributor	`stcosmosckrespaldos`
`id-github-deploy`	AcrPush, Reader	`acrcosmosckprod`
	Container Apps Contributor	`rg-cosmosck-prod`, `rg-cosmosck-qa`
	Credenciales federadas	`gh-produccion` y `gh-qa`: solo los environments `produccion` y `qa` del repo `Cosmos-Marcacion/cosmos`
`admin.azure@cosmos.com.pe`	Owner; Key Vault Secrets Officer	Suscripción; ambas bóvedas

- No hay contraseñas de servicio: GitHub entra a Azure por OIDC y las aplicaciones usan identidades administradas.
- Las bases de datos solo aceptan conexiones desde servicios de Azure. Para cargar datos se abre temporalmente la IP de origen (`instalar.sh ip <amb> abrir|cerrar`).
- La cuenta de respaldos no admite acceso público ni TLS inferior a 1.2.

7. Respaldos

- Imagen propia `mongo-backup:1` (mongodump y subida por la API REST de Blob con el token de la identidad, sin azcopy).
- Archivo por ejecución: `respaldos/cosmos-AAAAMMDD-HHMM.archive.gz`. Un volcado de menos de 1 KB se considera fallido y el job termina en error.
- Retención: se borran a los 90 días.
- **Aún no se ha ejecutado:** la base de producción está vacía hasta el corte; el primer respaldo de prueba se lanza tras migrar los datos.

8. Despliegue continuo (GitHub)

Elemento	Configuración
Workflow	`.github/workflows/deploy.yml`: push a `main` despliega en producción y a `develop` en QA
Gates de calidad	`qa.yml` (pruebas, cobertura, Semgrep). Si alguno falla no se despliega nada. Arreglados en el PR #1 (`fix/gates-qa`), 6/6 en verde.
Despliegue selectivo	Solo se despliega lo que cambió: backend, panel o ambos
Backend	Construye la imagen, la sube a ACR y ejecuta `deploy-revision.sh`: revisión nueva al 0 %, prueba de `/health/ready` (hasta 5 min), y solo entonces 100 % del tráfico. Si falla, el tráfico se queda en la anterior.
Panel	Node 22, `ng build` (configuración `production` o `qa`) y subida a Static Web Apps con `SWA_TOKEN`
Environments	`produccion` y `qa` creados, con `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `ACR_NAME`, `RG`, `APP_NAME` y el secreto `SWA_TOKEN`
Interruptor	Variable de repositorio `AZURE_DEPLOY_ENABLED=true`

Concurrencia	Un despliegue por rama a la vez; no se cancelan a medias

9. Código entregado (`infra/azure`)

Fichero	Función
`instalar.sh`	Instalador idempotente (se puede relanzar; respeta lo creado) y comandos de operación: `pro mover`, `ip`, `dominios`, `dns`, `corte`
`vars.sh`	Nombres, dominios, tiers y recursos
`documentdb.bicep`	Plantilla del cluster DocumentDB y su regla de firewall
`deploy-revision.sh`	Despliegue sin corte con vuelta atrás automática
`mongo-backup/`	Imagen del respaldo y del ping de QA
`conteos.js`	Conteo de documentos e índices por colección, para comparar origen y destino
`estado.sh`, `acceso.sh`	Diagnóstico de solo lectura: fase de la instalación y permisos efectivos
`DESPLIEGUE.md`, `README.md`	Guía paso a paso y referencia

10. Incidencias resueltas durante la instalación

#	Problema	Solución
1	La suscripción por defecto era `CosmosAzure_Dev`, con solo Contributor	Se eligió la suscripción Check-In
2	Sin rol en Check-In (CSP: el Owner lo tiene el partner)	Con el acceso elevado de administrador global se asignó Owner a la cuenta
3	`acrcosmosciprod` ocupado en otra suscripción	Prefijo cambiado a `cosmosck`
4	"Subscription not found" al comprobar el nombre de la cuenta de almacenamiento	El instalador registra los proveedores antes de comprobar nombres
5	`AuthorizationFailed` tras darse Owner	Token anterior a la asignación: `az logout` y `az login`
6	Cloud Shell sin almacenamiento persistente	Se reclonó el repo y se subieron los scripts a GitHub (commit `4fcdd76`)
7	Gates de QA fallando en `main` desde el 10/10 (zona horaria del runner, Semgrep, allowlists, cobertura de auth)	Corregido en el PR #1, 6/6 en verde

11. Decisiones respecto a la guía de aprovisionamiento (21/10/2026)

1. El panel llama a la API por su dominio y no por el proxy `/api` (límite de 45 s; las imágenes de formatos están en `/static`; los APK usan `apicheckin`).
2. Se conservan los dominios de producción (APK y WebAuthn).
3. Salud en `/health` y `/health/ready`.
4. Imagen en Debian, no Alpine (canvas, TensorFlow, LibreOffice).
5. 1 CPU / 2 GiB por réplica.
6. CORS en la API, no en el ingress (dos capas se pisan).
7. Identidades asignadas por el usuario, creadas y autorizadas antes de la app.

8. Un Key Vault por ambiente.
9. Respaldo por la API REST de Blob.
10. Un solo workflow para ambos ambientes, detrás de los gates.

12. Coste estimado

Precios de lista en East US 2, USD al mes, sin impuestos.

Recurso	USD/mes
API prod (1 CPU / 2 GiB, mínimo 1 réplica)	20 - 75
DocumentDB prod M10 (32 GB)	~18
Static Web Apps Standard (prod)	9
Container Registry Basic	~5
Log Analytics (5 GB/mes gratis)	0 - 10
Key Vault, respaldos, alertas, jobs	1 - 3
QA (API a 0 réplicas, base Free, SWA Free)	0 - 5
Total	**~60 - 125**

No incluye S3 (sigue en AWS), el servidor de Coolify durante el periodo de vuelta atrás ni GitHub Actions. Recomendado: presupuesto de 150 USD con alertas al 50, 80 y 100 %.

13. Estado y pendientes

#	Tarea	Responsable	Estado
1	Infraestructura en Azure (ambos ambientes)	Desarrollo	Hecho
2	Environments de GitHub y OIDC	Desarrollo	Hecho
3	Solicitud DNS 1 (QA, TXT y TTL a 300 s). A 04/11 la zona sigue sin cambios.	Área DNS (Optical IP)	Pendiente
4	Enlazar dominios de QA (`instalar.sh dominios qa`)	Desarrollo	Tras 3
5	Revisores requeridos en `produccion` y limitar ramas; fusionar PR #1	Desarrollo	Pendiente
6	Crear rama `develop` (primer despliegue de QA)	Desarrollo	Pendiente
7	Origen en Atlas 8.0 y destino creado en 7.0: validar compatibilidad en el ensayo de QA	Desarrollo	Pendiente
8	Copia de datos a QA, conteos y pruebas funcionales	Desarrollo	Pendiente
9	Despliegue previo a producción (sin tráfico real)	Desarrollo	Pendiente
10	Ventana de corte: migrar datos de prod y Solicitud DNS 2	Desarrollo y área DNS	Pendiente
11	Primer respaldo de prueba	Desarrollo	Tras 10
12	Presupuesto y alertas de CPU/almacenamiento de los clusters (80 %)	Desarrollo	Pendiente
13	Avisar al partner CSP y a TI del rol Owner; retirar el acceso elevado en `/`	Administrador Azure	Pendiente
14	Tras 7 a 14 días estables: dar de baja Coolify/AWS (excepto S3) y Netlify	Cosmos	Pendiente

14. Plan de corte y vuelta atrás

1. Comprobar el día anterior que el TTL de `checkin` y `apicheckin` ya es 300 s.
2. Avisar a los usuarios y detener (sin borrar) el backend en Coolify.
3. Volcar la base `cosmos` de Atlas (solo esa: el cluster es compartido con otros proyectos), restaurarla en `docdb-cosmosck-prod` y comparar conteos. **Si no coinciden, no seguir**: se arranca de nuevo Coolify.
4. Ejecutar `instalar.sh corte` y avisar al área DNS para la Solicitud 2. El script espera hasta 60 min a que el DNS apunte, enlaza los dominios y emite certificados.
5. Pruebas de humo: login, marcación desde la app sin reinstalar, reporte y PDF, subida de archivo.

Vuelta atrás (útil en las primeras horas; lo escrito en Azure después del corte no estará en Coolify): restaurar los dos registros DNS a sus valores actuales y arrancar el backend en Coolify. Netlify y Coolify se mantienen intactos durante 7 a 14 días.

15. Operación diaria

Necesidad	Cómo
Publicar en QA	Push a `develop`
Publicar en producción	PR a `main`, fusionar y aprobar en Actions
Volver a la versión anterior de la API	`az containerapp ingress traffic set -g rg-cosmosck-prod -n ca-cosmosck-api-prod --revision-weight <revisión-anterior>=100`
Ver registros	`az containerapp logs show -g rg-cosmosck-prod -n ca-cosmosck-api-prod --tail 100`
Cambiar una variable	`az containerapp update ... --set-env-vars CLAVE=valor` y luego `instalar.sh promover prod`
Cambiar un secreto	`az keyvault secret set --vault-name kv-cosmosck-prod ...` y reiniciar la revisión activa
Estado general	`PROY=cosmosck ./infra/azure/estado.sh` (exportar siempre `PROY=cosmosck`)

> **Repositorio**
> Despliega solo el monorepo de GitHub `Cosmos-Marcacion/cosmos`. Los repositorios de Bitbucket no están conectados: cada cambio hecho allí debe subirse también a GitHub para llegar a Azure.

Fuentes: registro del instalador (`cosmos-instalacion-20261030-215409.log`), scripts de `infra/azure`, notas de sesión del 28 y 30/10/2026 y consulta al DNS público del 04/11/2026.